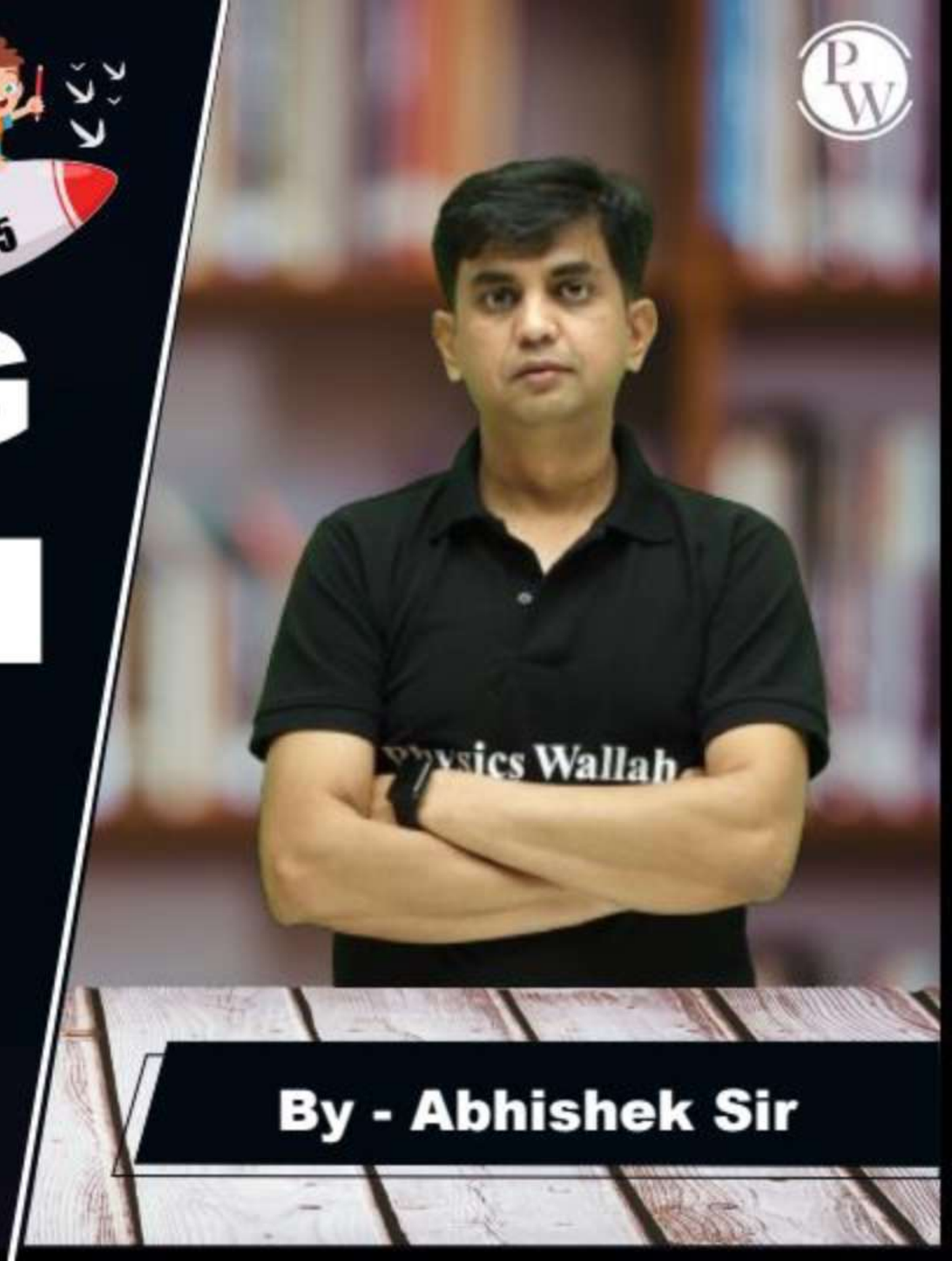
CS & IT ENGINEERING

Computer Network

Transport Layer

Lecture No. - 07



By - Abhishek Sir

Recap of Previous Lecture

Topic Socket Programming

Topic TCP Socket

Topics to be Covered

Topic UDP Socket

Topic TCP Flow Control

Topic TCP Congestion Control

ABOUT ME

Hello, I'm Abhishek

- GATE CS AIR - 96
- M.Tech (CS) - IIT Kharagpur
- 12 years of GATE CS teaching experience

Telegram Link : https://t.me/abhisheksirCS_PW

@ABHISHEKSIRCS_PW

#Q. A client process P needs to make a TCP connection to a server process S. Consider the following situation: the server process S executes a socket(), a bind() and a listen() system call in that order, following which it is preempted. Subsequently, the client process P executes a socket() system call followed by connect() system call to connect to the server process S. The server process has not executed any accept() system call. Which one of the following events could take place?

[GATE-2008]

(A) connect() system call returns successfully

(B) connect() system call blocks

(C) connect() system call returns an error

(D) connect() system call results in a core dump

Ans: C

P (client)

socket()

connect()

SYN (Conn. Req.)

S (Server)

socket()

Bind()

Listen()

Conn. Req.

1. Block
2. Queue

Timeout

[Return -1;
Error]

Topic : Socket Programming with TCP

TCP Client		TCP Server
socket()		socket() 1
connect()	SYN →	bind() 2
		listen() 3
		accept()
	← SYN+ACK	
	ACK →	
write()	DATA segment →	read()
read()	← DATA segment	write()
close()	← 4-way handshaking →	close()

Topic : Server Type

→ Several clients can request service of the server in the same time.

Non-concurrent Server :

→ Service one client at one time

→ Other client request must wait

Concurrent Server :

→ Services all client requests simultaneously

Topic : Socket Programming with TCP

Non-concurrent TCP Server :-

```
socket()
bind()

while(1) {
        listen()
        accept()
        read() / write()   }

close()
```

Topic : Socket Programming with TCP

Concurrent TCP Server :-

```
socket()
bind()
while(1) {
        listen()
        if ( fork() == 0 ) {
                accept()
                read() / write()
                close()   }   }

close()
```

Topic : Send a message on socket

```
#include <sys/socket.h>

ssize_t sendto ( int socket, const void *message, size_t length, int flags,
                 const struct sockaddr *dest_addr, socklen_t dest_len );
```

sockfd

(Dest. IP + Port)

sockfd : file descriptor of the socket

return : On success, the number of bytes send is returned
On error, -1 is returned

Topic : Receive a message from a socket

```
#include <sys/socket.h>

ssize_t recvfrom ( int socket, void *buffer, size_t length, int flags,
                   struct sockaddr *address, socklen_t address_len );
```

sockfd

(Source IP + Port)

sockfd : file descriptor of the socket

return : On success, the number of bytes receive is returned
On error, -1 is returned

Topic : Socket Programming with UDP

UDP Client		UDP Server
socket()		socket()
sendto()	UDP Packet →	bind()
		recvfrom()
		sendto()
	← UDP Packet	
recvfrom()		close()
close()		

Topic : Connected UDP sockets

UDP Client	UDP Server
socket()	socket()
connect()	Bind()
write()	
	read()
	write()
read()	close()
close()	

#Q. Consider socket API on a Linux machine that supports connected UDP sockets. A connected UDP socket is a UDP socket on which connect function has already been called. Which of the following statements is/are CORRECT?

I. A connected UDP socket can be used to communicate with multiple peers simultaneously.

II. A process can successfully call connect function again for an already connected UDP socket.

[GATE-2017, Set-2]

IIT-R

H.W

(A) I only

(B) II only

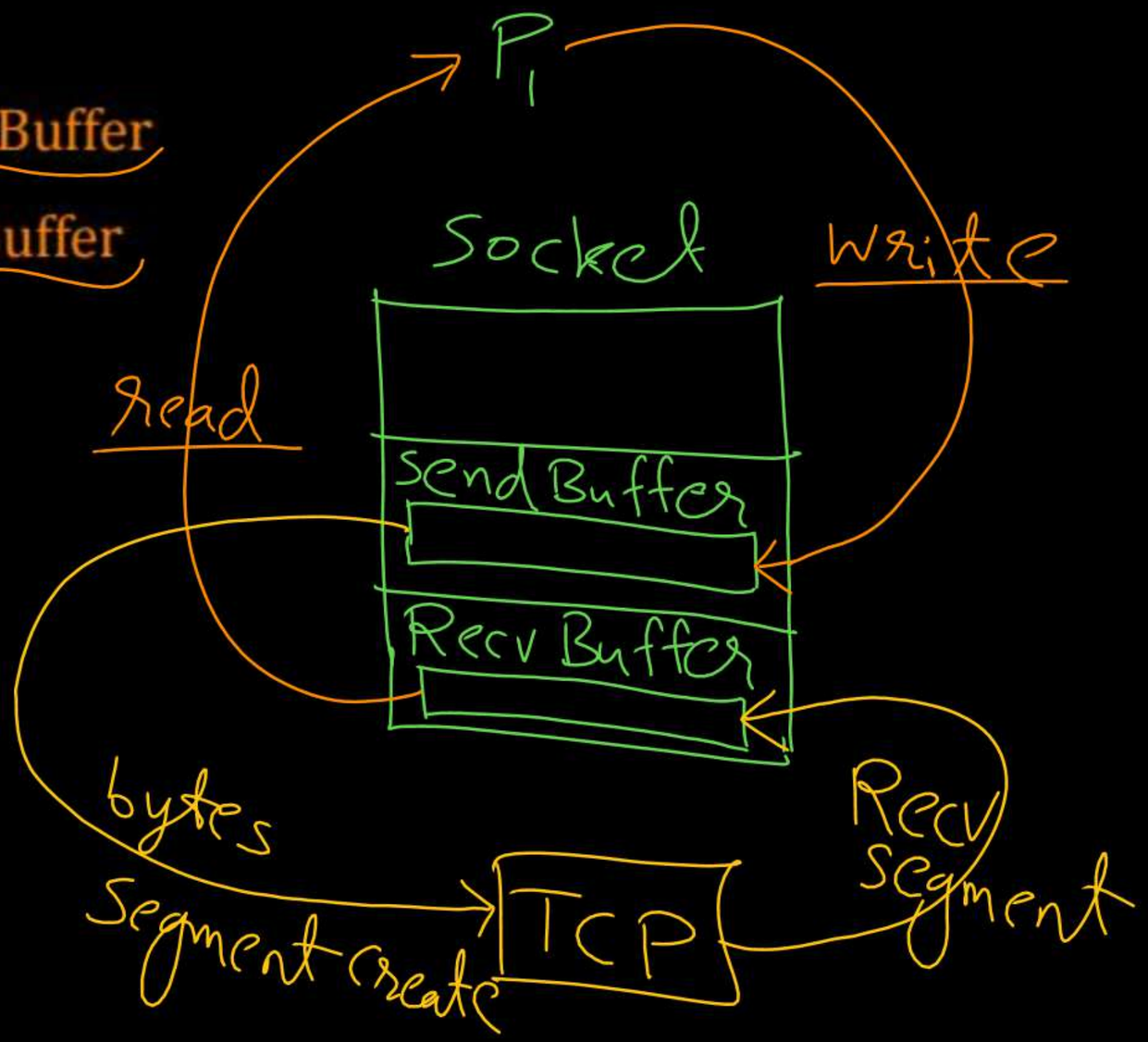
(C) Both I and II

(D) Neither I nor II

Topic : TCP Buffer

→ Each TCP socket have two buffers :-

1. Send buffer size : MaxSendBuffer
2. Receive buffer size : MaxRcvBuffer

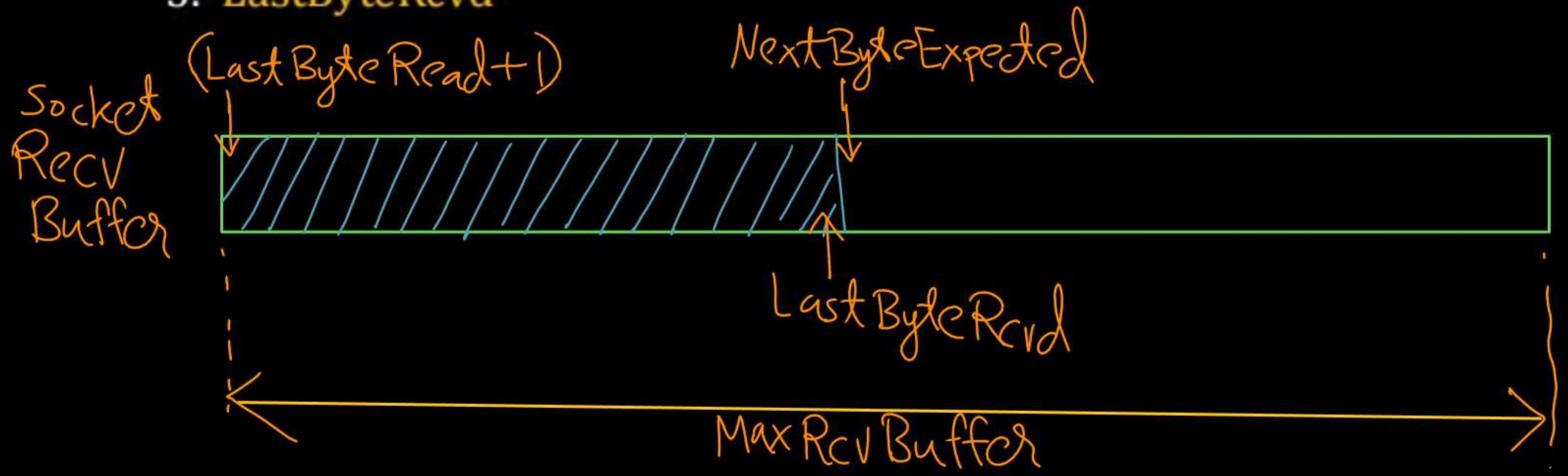


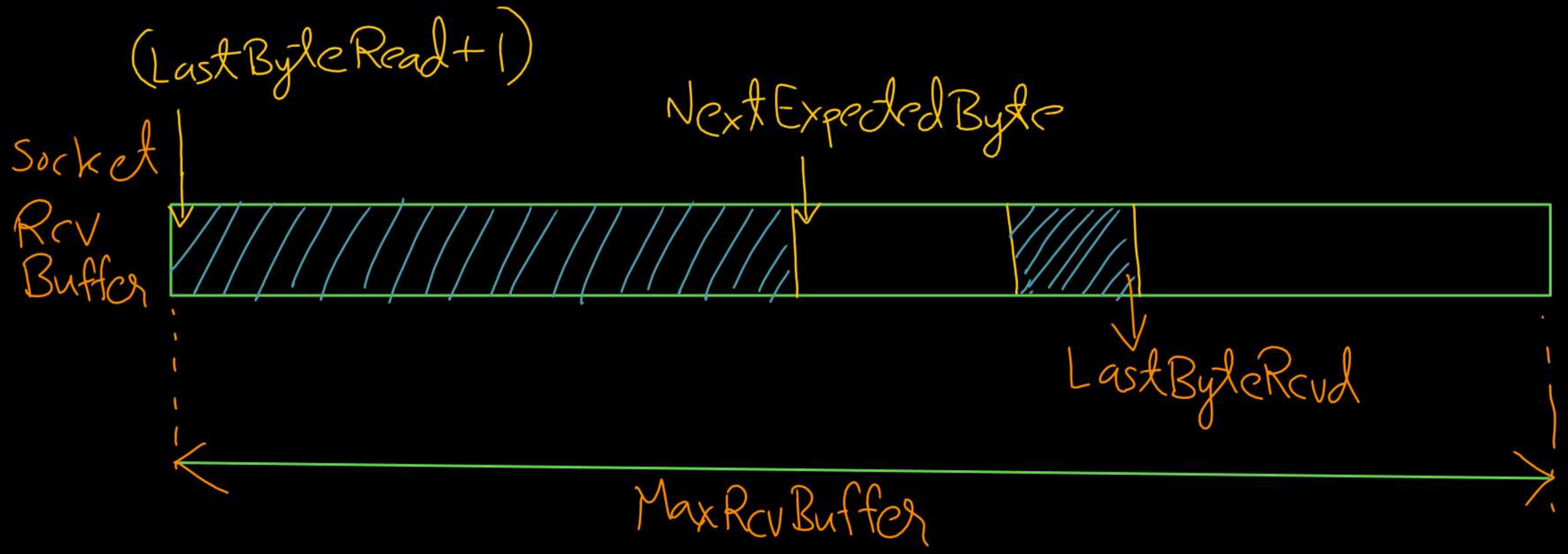
Topic : TCP Flow Control

→ Receive buffer size : MaxRcvBuffer

→ TCP receiver maintain three variables :-

1. LastByteRead
2. NextByteExpected
3. LastByteRcvd




(LastByteRead+1)
NextExpectedByte
Socket
Rcv
Buffer
LastByteRcvd
MaxRcvBuffer

Topic : TCP Flow Control

→ Receive buffer size : MaxRcvBuffer

→ TCP receiver maintain three variables :-

1. LastByteRead
2. NextByteExpected
3. LastByteRcvd

$$\text{LastByteRead} < \text{NextByteExpected} \leq (\text{LastByteRcvd} + 1)$$

Topic : TCP Flow Control

→ Receive buffer size : MaxRcvBuffer

→ TCP receiver maintain three variables :-

1. LastByteRead
2. NextByteExpected
3. LastByteRcvd

$$[\text{LastByteRcvd} - \text{LastByteRead}] \leq \text{MaxRcvBuffer}$$

Always TRUE

$$\text{AdvertisedRcvWindow} = \text{MaxRcvBuffer} - [\text{NextByteExpected} - (\text{LastByteRead} + 1)]$$

Topic : TCP Segment Structure

0	16	31
Source Port No. (16 bit)	Dest. Port No. (16 bit)	
Sequence Number (32 bit)		
Acknowledgement Number (32 bit)		
HLEN	URG ACK PSH RST SYN FIN	Window Size (16 bit)
Checksum (16 bit)		Urgent Pointer (16 bit)
Options [Optional Header]		
—//— Payload —//—		

TCP Header (5 to 15 word) (20 to 60 byte)

TCP Base Header 5 word (20 byte)

0 to 10 word (0 to <40 byte)

HLEN →

Topic : Window Size

→ Receiver Window Size (16-bit)

→ An advertisement from receiver

→ Define how much data (in bytes) the receiving device can accept

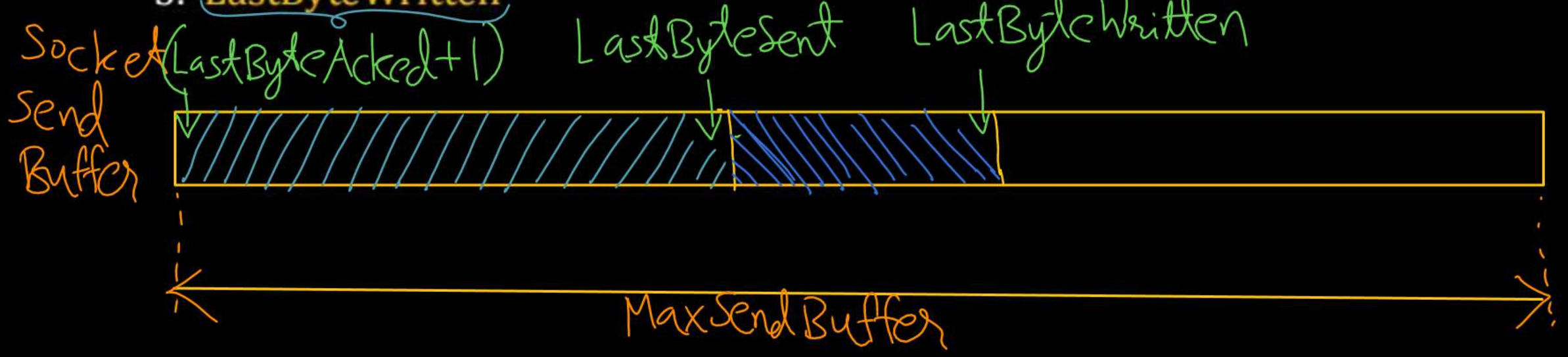
→ Define current available space (in bytes) in receiver's receiving window

Topic : TCP Flow Control

→ Send buffer size : MaxSendBuffer

→ TCP sender maintain four variables :-

1. RWND = Receiver Advertised Window Size
2. LastByteAcked
2. LastByteSent
3. LastByteWritten



Topic : TCP Flow Control

→ Send buffer size : MaxSendBuffer

→ TCP sender maintain four variables :-

1. RWND = Receiver Advertised Window Size
2. LastByteAcked
2. LastByteSent
3. LastByteWritten

$$\boxed{\text{LastByteAcked} \leq \text{LastByteSent} \leq \text{LastByteWritten}}$$

Topic : TCP Flow Control

→ Send buffer size : MaxSendBuffer

→ TCP sender maintain four variables :-

1. RWND = Receiver Advertised Window Size
2. LastByteAcked
2. LastByteSent
3. LastByteWritten

$$\boxed{(\text{LastByteWritten} - \text{LastByteAcked}) \leq \text{MaxSendBuffer}}$$

Always TRUE

→ The process wants to write X bytes in 'Send Buffer'

if [(LastByteWritten – LastByteAcked) + X] > MaxSendBuffer

→ Not allowed to write

Topic : TCP Flow Control

→ Send buffer size : MaxSendBuffer

→ TCP sender maintain four variables :-

1. RWND = Receiver Advertised Window Size
2. LastByteAcked
2. LastByteSent
3. LastByteWritten

$$\boxed{(\text{LastByteSent} - \text{LastByteAcked}) \leq \text{RWND}}$$

Flow Control Condition

→ TCP wants to send a segment contains Y bytes in payload

if [(LastByteSent – LastByteAcked) + Y] > RWND

→ Not allowed to send

Topic : TCP Flow Control

→ Send buffer size : MaxSendBuffer

→ TCP sender maintain four variables :-

1. RWND = Receiver Advertised Window Size
2. LastByteAcked
2. LastByteSent
3. LastByteWritten

$$\boxed{\text{EffectiveWindow} = \text{RWND} - (\text{LastByteSent} - \text{LastByteAcked})}$$

2 mins Summary

Topic UDP Socket

Topic TCP Flow Control

Topic [redacted]

THANK - YOU